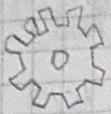


40's

WW2 → Alan Turing



Mechanical Computing Machine

John von Neumann

$f$  encoding

MACHINE CODE

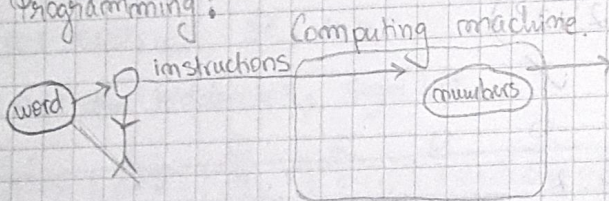
$f^{-1}$  decoding

BYTE CODE

a more effective machine:

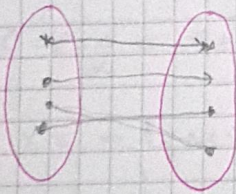
Thomas Edison light bulbs → transistors

Programming:



Program: a finite succession of instructions

$A \times B \rightsquigarrow$  cartesian product (Cartesius → René Descartes)



$$A \times B \ni R \ni f \ni f^{-1}$$

$$\{0,1\} \times \{0,1\} = \{0,1\}$$

$$= \{(0,0), (0,1), (1,0), (1,1)\}$$

$$= \begin{matrix} \downarrow & \downarrow & \downarrow & \downarrow \\ = & < & > & = \end{matrix}$$

$f$  bij.  $\rightarrow \exists f^{-1}$

byte: domeniul sau algebric:  $\{0,1\} \times \{0,1\} \dots \times \{0,1\} =$

$$= 2^8 = 256$$

8

NOV instructiune către machine code. (instruction code)

atribui fiecare cod între 0,255 .....

ADD

SUB

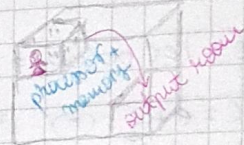
50's

TV, STORAGE

Transistor → Logical Gates  
(AT&T)

Electromechanical

ENIAC



George Boole  
(19th Britain)

Boolean Algebra  
 $\{0,1\}^*$

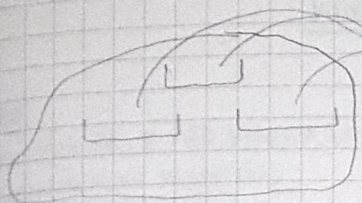
Claude Shannon

old logic  
Aristotel logic

transistor (R)

cea mai luminată  
minte până la Newton



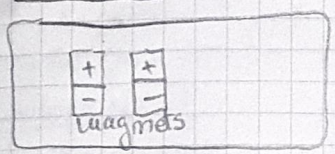


formal parameters.  
 => CALL PROCEDURE => return result.

PROCEDURE NAME

storage was the hardware needed to store that procedure.

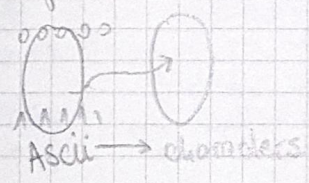
STORAGE



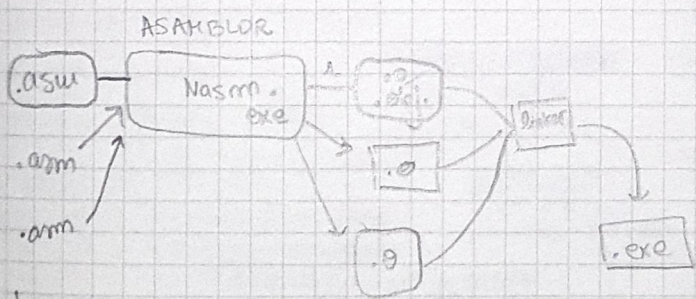
=> STORED PROCEDURE  
 => Procedural Programming *paradigm*  
 ↓  
 Modular programming  
 (Appollo 11 github) 60' NASA

.asm .agc → text file

File = finite succession of bytes.



start:  
 ...  
 call [exit]  
 CALL STACK



• h → abstraction  
 • c → implementation

40's

Grace Hopper → Marking (warime) US army US navy  
 → Compiler Cobol.

AT & T Bell Lab } Dennis Richie  
 } Ken Thompson → **C**

*Data Structures and Algorithms*

UNIX (First OS)

↳ Arpanet

C++  
 everything is an object

80's

=> Paradigm Shift.

x3ES